

COSC 3P32 - File and Database Systems

Assignment #3 - Solutions

Question 1 [4 marks]

Relation: Assignments(cid, semester, aid, type, duedate, weight, maxmarks)

Codes: C = cid, S = semester, A = aid, T = type, D = duedate, W = weight, M = maxmarks

a) The key (cid, semester, aid) determines all remaining attributes:

CSA → TDWM

b) There is only one assignment for a given course, semester, and due date:

CSD → A

c) Same weight for a given course and semester implies the same max marks:

CSW → M

d) Same type for the same course implies the same weight:

CT → W

Question 2 [20 marks]

Schema: R = VWXYZ

FDs: F = { VW→XY, V→X, W→Z, YZ→Z, Z→V }

Part a) Three independent reasons the table is not a legal instance [6 marks]

V	W	X	Y	Z
v1	w1	x1	y1	z1
v2	w1	x2	y2	z1
v2	w2	x1	y2	z2

Reason 1 - V→X violated (rows 2 and 3).

Both rows have V = v2, yet X = x2 in row 2 and X = x1 in row 3. The FD V→X requires a unique X value for every V value.

Reason 2 - Z→V violated (rows 1 and 2).

Both rows have Z = z1, yet V = v1 in row 1 and V = v2 in row 2. The FD Z→V requires a unique V value for every Z value.

Reason 3 - W→V violated (rows 1 and 2).

W→Z and Z→V together imply W→V (a member of F⁺). Both rows have W = w1, yet V = v1 in row 1 and V = v2 in row 2, violating this derived dependency.

Part b) FDs on $R_1 = VWX$ and $R_2 = WYZ$ [4 marks]

$R_1 = VWX$

Project F onto $\{V, W, X\}$ by computing closures:

- $V^+ \cap VWX = \{V, X\} \rightarrow V \rightarrow X$
- $W^+ = \{W, Z, V, X\}$ (via $W \rightarrow Z, Z \rightarrow V, V \rightarrow X$); $W^+ \cap VWX = \{V, W, X\} \rightarrow W \rightarrow V$ ($W \rightarrow X$ is implied by $W \rightarrow V + V \rightarrow X$ and not listed separately)

Non-trivial FDs on R_1 (not trivial, not implied by others in the set):

$V \rightarrow X, W \rightarrow V$

$R_2 = WYZ$

Project F onto $\{W, Y, Z\}$:

- $W^+ = VWXYZ$ ($W \rightarrow Z, Z \rightarrow V, V \rightarrow X$, then $VW \rightarrow XY$ adds Y); $W^+ \cap WYZ = WYZ \rightarrow W \rightarrow Z$ and $W \rightarrow Y$
- $Z^+ \cap WYZ = \{Z\}$; $Y^+ \cap WYZ = \{Y\}$; $YZ^+ \cap WYZ = \{Y, Z\} \rightarrow$ all trivial

Non-trivial FDs on R_2 :

$W \rightarrow Z, W \rightarrow Y$ (equivalently $W \rightarrow YZ$)

Part c) Is the decomposition lossless-join? [2 marks]

$R_1 \cap R_2 = \{W\}$.

W^+ under F = VWXYZ, so $W \rightarrow VWX = R_1$. Because the shared attribute W is a superkey of R_1 , the decomposition is **lossless-join**. ✓

Part d) Lossless-join BCNF decomposition [5 marks]

Step 0 - Find the candidate key of R.

W^+ : $W \rightarrow (W \rightarrow Z) \rightarrow Z \rightarrow (Z \rightarrow V) \rightarrow V \rightarrow (V \rightarrow X) \rightarrow X$, and with both V, W available $VW \rightarrow (VW \rightarrow XY) \rightarrow Y$. $W^+ = VWXYZ = R$.

Unique candidate key of R: **W**.

Step 0 - Identify BCNF violations.

- $VW \rightarrow XY$: $VW \supseteq \{W\}$ (key) $\rightarrow$ superkey ✓
- **$V \rightarrow X$** : $V^+ = VX \neq R \rightarrow$ violation
- $W \rightarrow Z$: W is key $\rightarrow$ superkey ✓
- $YZ \rightarrow Z$: trivial ✓
- **$Z \rightarrow V$** : $Z^+ = VXZ \neq R \rightarrow$ violation

Step 1 - Decompose using $V \rightarrow X$ ($V^+ = \{V, X\}$).

$R_A = VX$ (key: V; FD: $V \rightarrow X$; BCNF ✓)

$R_B = VWYZ$ (key: W; BCNF violation: $Z \rightarrow V$ because $Z^+ \cap VWYZ = VZ \neq VWYZ$)

Step 2 - Decompose $R_B = VWYZ$ using $Z \rightarrow V$ ($Z^+ \cap VWYZ = \{V, Z\}$).

$R_C = VZ$ (key: Z; FD: $Z \rightarrow V$; BCNF ✓)

$R_D = WYZ$ (key: W; FDs: $W \rightarrow Z, W \rightarrow Y$; both have superkey on LHS $\rightarrow$ BCNF ✓)

Final BCNF decomposition:

Relation	Key	FDs
VX	V	$V \rightarrow X$
VZ	Z	$Z \rightarrow V$
WYZ	W	$W \rightarrow Z, W \rightarrow Y$

Part e) Is the BCNF decomposition dependency-preserving? [3 marks]**Compute the minimal cover of F:**

1. Decompose RHS: $\{ VW \rightarrow X, VW \rightarrow Y, V \rightarrow X, W \rightarrow Z, YZ \rightarrow Z, Z \rightarrow V \}$
2. Remove trivial FD $YZ \rightarrow Z$.
3. Minimize LHS of $VW \rightarrow X$: W^+ (without $VW \rightarrow X$) = $\{W, Z, V, X\}$; $X \in W^+ \rightarrow V$ redundant $\rightarrow$ replace with $W \rightarrow X$.
4. Minimize LHS of $VW \rightarrow Y$: W^+ (without $VW \rightarrow Y$) = $\{W, Z, V, X\}$; $Y \notin W^+ \rightarrow V$ is not redundant. $VW \rightarrow Y$ stays.
5. Remove redundant FDs: $W \rightarrow X$ is implied by $W \rightarrow Z, Z \rightarrow V, V \rightarrow X \rightarrow$ remove $W \rightarrow X$.

Minimal cover: $\{ VW \rightarrow Y, V \rightarrow X, W \rightarrow Z, Z \rightarrow V \}$

FD	Preserved in	Without join?
$V \rightarrow X$	VX	✓
$W \rightarrow Z$	WYZ	✓
$Z \rightarrow V$	VZ	✓
$VW \rightarrow Y$	WYZ has $W \rightarrow Y$; augmentation gives $VW \rightarrow Y$	✓

All FDs in the minimal cover are preserved without joining. The decomposition **is dependency-preserving**. ✓

Question 3 [11 marks]

Schema: $R = ABCDEGH$

FDs: $F = \{ B \rightarrow AGH, CD \rightarrow B, C \rightarrow EG, E \rightarrow AD, G \rightarrow E \}$

Part a) Minimal cover [6 marks]**Step 1 - Decompose all RHS into singletons:**

$\{ B \rightarrow A, B \rightarrow G, B \rightarrow H, CD \rightarrow B, C \rightarrow E, C \rightarrow G, E \rightarrow A, E \rightarrow D, G \rightarrow E \}$

Step 2 - Minimize left-hand sides.

Only $CD \rightarrow B$ has a multi-attribute LHS.

- Remove C? $D^+ = \{D\}$. $B \notin D^+ \rightarrow C$ is not redundant.
 - Remove D? C^+ (without $CD \rightarrow B$) = $\{C, E, G, A, D\}$. $B \notin C^+ \rightarrow D$ is not redundant.
- $CD \rightarrow B$ stays unchanged.

Step 3 - Remove redundant FDs:

- $B \rightarrow A$: $B^+ = \{B, G, H, E, A, D\}$ (via $B \rightarrow G \rightarrow E \rightarrow A$). $A \in B^+ \rightarrow$ **redundant, remove**.
- $B \rightarrow G$: B^+ without $B \rightarrow G = \{B, H\}$. $G \notin \rightarrow$ not redundant.
- $B \rightarrow H$: B^+ without $B \rightarrow H = \{B, G, E, A, D\}$. $H \notin \rightarrow$ not redundant.
- $CD \rightarrow B$: $(CD)^+$ without $CD \rightarrow B = \{C, D, E, G, A\}$. $B \notin \rightarrow$ not redundant.
- $C \rightarrow E$: C^+ without $C \rightarrow E = \{C, G, E, A, D\}$ (via $C \rightarrow G \rightarrow E$). $E \in C^+ \rightarrow$ **redundant, remove**.
- $C \rightarrow G$: C^+ without $C \rightarrow G = \{C\}$. $G \notin \rightarrow$ not redundant.
- $E \rightarrow A$: E^+ without $E \rightarrow A = \{E, D\}$. $A \notin \rightarrow$ not redundant.
- $E \rightarrow D$: E^+ without $E \rightarrow D = \{E, A\}$. $D \notin \rightarrow$ not redundant.
- $G \rightarrow E$: G^+ without $G \rightarrow E = \{G\}$. $E \notin \rightarrow$ not redundant.

Minimal cover:

$\{ B \rightarrow GH, CD \rightarrow B, C \rightarrow G, E \rightarrow AD, G \rightarrow E \}$

(individual FDs: $B \rightarrow G, B \rightarrow H, CD \rightarrow B, C \rightarrow G, E \rightarrow A, E \rightarrow D, G \rightarrow E$)

Part b) Candidate keys [2 marks]

RHS attributes in the minimal cover: $\{G, H, B, A, D, E\}$.

Attribute **C** never appears on any RHS $\rightarrow$ C must be in every candidate key.

Compute C^+ :

$C \rightarrow (C \rightarrow G) \rightarrow G \rightarrow (G \rightarrow E) \rightarrow E \rightarrow (E \rightarrow A, E \rightarrow D) \rightarrow A, D$. Now C and D are in the closure $\rightarrow (CD \rightarrow B) \rightarrow B \rightarrow (B \rightarrow H) \rightarrow H$.

$C^+ = ABCDEGH = R$. ✓

C alone is a superkey; no proper subset is a superkey. **Unique candidate key: $\{ C \}$**

Part c) 3NF Synthesis [3 marks]

Step 1 - Create one relation per FD group in the minimal cover:

Relation	Generating FD(s)	Key
BGH	$B \rightarrow GH$	B
BCD	$CD \rightarrow B$	CD (also C alone, since $C \rightarrow D$ holds via the chain $C \rightarrow G \rightarrow E \rightarrow D$)
CG	$C \rightarrow G$	C
ADE	$E \rightarrow AD$	E
GE	$G \rightarrow E$	G

Step 2 - Ensure a candidate key of R is present.

The unique candidate key is $\{C\}$. Relation BCD contains C $\rightarrow$ no extra key relation needed.

✓

Step 3 - Remove redundant relations.

No relation is a subset of any other. None removed.

Final 3NF decomposition: { BGH, BCD, CG, ADE, GE }

Relation	Non-trivial FDs (not implied by others in the same set)
BGH	$B \rightarrow GH$
BCD	$C \rightarrow BD$, $B \rightarrow D$ (C is the sole candidate key of BCD; $C \rightarrow D$ holds via $C \rightarrow G \rightarrow E \rightarrow D$; $B \rightarrow D$ holds via $B \rightarrow G \rightarrow E \rightarrow D$ proj $CD \rightarrow B$ is implied by $C \rightarrow B$, a part of $C \rightarrow BD$)
CG	$C \rightarrow G$
ADE	$E \rightarrow AD$
GE	$G \rightarrow E$

Lossless-join: BCD contains C (the sole candidate key of R). The 3NF synthesis algorithm guarantees a lossless-join decomposition. ✓

Dependency-preserving: Every FD in the minimal cover is directly represented in one of the relations ($B \rightarrow GH \in \text{BGH}$, $CD \rightarrow B \in \text{BCD}$, $C \rightarrow G \in \text{CG}$, $E \rightarrow AD \in \text{ADE}$, $G \rightarrow E \in \text{GE}$). ✓

Question 4 [15 marks]

Schema: Restaurant(rname, address, phone, stars) Chef(cname, specialdish, rating)
CooksFor(cname, rname, salary) Offers(rname, dishname, price)

Part a) CHECK: chef rating between 0 and 10 [2 marks]

Add to the **Chef** table definition:

```
CHECK (rating >= 0 AND rating <= 10)
```

Part b) CHECK: no chef salary at 'BigMac' exceeds 100 [2 marks]

Add to the **CooksFor** table definition:

```
CHECK (rname <> 'BigMac' OR salary <= 100)
```

Part c) Assertion: every restaurant has at least one chef [3 marks]

```
CREATE ASSERTION every_restaurant_has_chef
CHECK (
    NOT EXISTS (
        SELECT *
        FROM Restaurant R
        WHERE NOT EXISTS (
            SELECT *
            FROM CooksFor CF
            WHERE CF.rname = R.rname
        )
    )
);
```

Part d) Assertion: every offered dish has a chef whose special dish it is [3 marks]

```
CREATE ASSERTION every_dish_has_special_chef
CHECK (
    NOT EXISTS (
        SELECT *
        FROM Offers O
        WHERE NOT EXISTS (
            SELECT *
            FROM Chef C
            WHERE C.specialdish = O.dishname
        )
    )
);
```

Part e) Trigger: 10% salary raise when stars increase [5 marks]

```
CREATE TRIGGER stars_raise
AFTER UPDATE OF stars ON Restaurant
REFERENCING OLD ROW AS old_row
              NEW ROW AS new_row
FOR EACH ROW
WHEN (new_row.stars > old_row.stars)
BEGIN ATOMIC
    UPDATE CooksFor
    SET     salary = salary * 1.10
    WHERE  rname = new_row.rname;
END;
```

The **WHEN** clause ensures the trigger fires only when the star rating increases. The **FOR EACH ROW** granularity makes the updated restaurant's rname available via new_row. The 10% raise is applied to every chef currently listed in CooksFor for that restaurant.